

Roteiro de apresentação: TP1 de Recuperação de Informação

Sep 30, 2026 · @Umed

O trabalho tem uma tese única, e ela deve abrir e fechar a apresentação: nesta coleção, quase toda a discordância entre o modelo vetorial e o BM25 vem de um só mecanismo, a forma como cada um normaliza o comprimento do documento. Tudo o mais são evidências dessa afirmação.

Divisão e roteiro

A divisão segue a fronteira natural do código: quem constrói o ranking e quem mede o ranking. Victor fica com os cinco módulos que vão do arquivo bruto ao score. João Pedro fica com os módulos que avaliam esse score e extraem as conclusões.

Cada um domina a sua metade, mas os dois precisam saber responder o básico da outra. O enunciado prevê avaliação individual, e o professor pode pedir a qualquer um para explicar uma fórmula ou justificar o score de um documento.

Bloco	Quem	Min	Cobre
1. Problema e tese	Victor	2	Cranfield, os dois modelos, a tese do comprimento
2. Dados e pré-processamento	Victor	3	<code>dataset.py</code> , <code>preprocess.py</code> , requisito 1
3. Os dois modelos	Victor	4	<code>indexing.py</code> , <code>vsm.py</code> , <code>bm25.py</code> , requisitos 2 e 3
4. Como medimos	João Pedro	2	<code>metrics.py</code> , requisito 4
5. Comparação e análise por consulta	João Pedro	3	requisitos 5 e 6
6. Parâmetros, reformulação e erros	João Pedro	4	requisitos 7, 8 e 9

Bloco	Quem	Min	Cobre
7. Fechamento	os dois	1	volta à tese, limites do casamento léxico

São 19 minutos. Se o tempo for menor, corte primeiro o bloco 6, mantendo só a reformulação de q219, que é o exemplo mais forte.

Victor: do texto bruto ao ranking

A sua fala tem um fio condutor: cada módulo resolve um problema que o anterior deixou em aberto. Termine mostrando que os dois modelos saem do mesmo índice, porque isso é o que torna a comparação justa.

dataset.py: ler a coleção sem desalinhar

Baixa o tarball da University of Glasgow, confere o SHA-256 e faz o parsing do formato SMART. São 1400 documentos, 225 consultas e 1837 julgamentos.

Duas armadilhas valem ser contadas, porque mostram leitura cuidadosa da coleção:

1. **Os ids das consultas não são sequenciais.** O campo `.I` de `cran.qry` vale 001, 002, 004, 008 até 365, mas o `cranqrel` numera de 1 a 225 pela posição no arquivo. Usar o `.I` como chave desalinha tudo silenciosamente e derruba as métricas. `load_queries` renumera por posição e guarda o original em `Query.original_id`.
2. **A escala de relevância é invertida.** Grau 1 é o melhor e 4 é o pior; -1 significa sem interesse.

Há ainda `Document.text`, que decide o que indexar: em 1395 dos 1400 documentos o campo `.W` já começa com o título, então concatenar `.T` dobraria o `tf` dos termos do título. Só concatenamos quando o título ainda não é prefixo do resumo.

preprocess.py: as quatro configurações do requisito 1

A classe `Preprocessor` é um pipeline configurável com dois interruptores, stopwords e stemming, o que dá as quatro combinações exigidas em `PREPROCESSING_CONFIGS`.

Três decisões para defender:

- **Tokenização por [a-z]+**, sobre o texto já em minúsculas. É determinística e serve ao texto ASCII técnico do Cranfield. O efeito colateral importa: hifens viram separadores, então `photo-thermoelastic` gera `photo` e `thermoelast`. Isso reaparece na reformulação de q15.
- **Stopwords antes do stemming.** A lista do NLTK é definida sobre formas plenas, e o radical de uma stopwords pode coincidir com o de um termo de conteúdo.
- **As stopwords do NLTK têm apóstrofo** (`don't`), mas o tokenizador só produz letras. Por isso cada entrada é expandida em fragmentos alfabéticos, e a lista vai a disco em `data/stopwords_en.txt` para a execução ser reproduzível sem rede.

O resultado está na Tabela 1 e na Figura 1 do relatório. O MAP do BM25 sobe de 0,269 para 0,305, ganho de 13,2%, e o ponto da sua fala é **por que os dois efeitos se somam**: stopwords quase não mudam o vocabulário (7044 para 6925) mas eliminam 42% dos tokens, mexendo no denominador das normalizações; o stemming não remove token algum, mas funde formas e corta 38% do vocabulário, mexendo na cobertura do casamento léxico.

indexing.py: um índice, dois modelos

O coração é uma matriz esparsa CSR de 1400 por 4292 com frequências brutas. Ela é guardada também em CSC, porque varrer a lista de postings de um termo é o acesso natural do BM25, enquanto o vetor de um documento é o acesso natural do CSR.

Diga a frase que justifica o experimento inteiro: **os dois modelos derivam seus pesos da mesma matriz**, logo qualquer diferença de desempenho vem da função de ranqueamento, nunca da indexação.

Um detalhe fácil de errar e bom de saber: `df` é calculado com `np.diff(tf.tocsc().indptr)`, ou seja, conta em quantos documentos o termo aparece, não quantas vezes ocorre no total.

vsm.py: TF-IDF e cosseno

O esquema é o `ltc.ltc` da Aula 04, igual para documento e consulta:

$$w_{\{i,j\}} = (1 + \log_{10} f_{\{i,j\}}) \times \log_{10} \frac{N}{n_i}$$

No `__post_init__` os pesos são calculados sobre `weights.data`, isto é, só nos pares documento-termo com frequência positiva, e depois cada documento é normalizado para norma L2 unitária. **Como os dois vetores já saem unitários, o cosseno vira um produto**

interno. É o mesmo cálculo, reorganizado para rodar de uma vez sobre os 1400 documentos.

Saiba explicar que `idf` vale exatamente zero para um termo presente em toda a coleção, que assim é eliminado do ranking.

bm25.py: implementado termo a termo

$$\mathrm{score}(d, q) = \sum_{t \in q} \mathrm{IDF}(t) \cdot \frac{f_{t,d}(k_1+1)}{f_{t,d} + k_1 \left(1 - b + b \frac{|d|}{\mathrm{avgdl}}\right)}$$

O enunciado proíbe implementação pronta que esconda o score, então mostre o laço de `score()` : para cada termo da consulta ele pega as postings e acumula a contribuição só nos documentos onde o termo ocorre. Documentos sem nenhum termo da consulta ficam com score zero, como esperado.

Três pontos de defesa:

- `length_norm` é pré-computado por documento, porque $k_1(1 - b + b|d|/\mathrm{avgdl})$ não depende do termo.
- Com `k1 = 0` o modelo vira binário, e existe um teste que verifica isso.
- Usamos o IDF de Robertson suavizado. O `1 +` impede peso negativo para termos em mais da metade da coleção, o que importa numa coleção temática onde `flow` ocorre em 730 dos 1400 documentos.

Se sobrar tempo, conte a identidade algébrica: a forma de Robertson é idêntica a $\log((N+1)/(n+0,5))$ e a do BIM é $\log((N+0,5)/(n+0,5))$, diferindo por uma constante de 0,00036. Por isso o MAP mudou só na sexta casa decimal. É um resultado algébrico, não empírico.

O desempate

Ambos os modelos terminam em `_rank_from_scores`, que usa `np.lexsort` para ordenar por score decrescente e desempatar por `doc_id` crescente. É o que garante que duas execuções produzam rankings idênticos.

João Pedro: medir e explicar

O fio da sua parte é outro: o enunciado diz que não basta informar qual modelo teve a maior média. Cada experimento seu existe para sustentar uma afirmação com exemplo concreto.

metrics.py: tudo implementado do zero

Seis métricas, nenhuma biblioteca: `precision_at_k`, `recall_at_k`, `f1_at_k`, `average_precision`, `reciprocal_rank`, `ndcg_at_k`, mais `interpolated_precision_recall` em 11 pontos.

Quatro detalhes que o professor pode cobrar:

- **O denominador da AP é o total de relevantes**, não o de relevantes recuperados. Assim, relevante não recuperado entra como zero e a métrica penaliza baixa revocação.
- **P@10 divide por k fixo**, não pelo tamanho do ranking.
- **A interpolação** segue o slide 20 da Aula 05: a precisão em cada nível é o máximo das precisões em revocações iguais ou maiores.
- **GainMapping.INVERTED** corrige a escala do Cranfield para o NDCG, mapeando 1 para 4, 2 para 3, 3 para 2 e 4 para 1. Usar o grau bruto daria o maior ganho ao documento menos relevante, e a diferença não é cosmética: NDCG@10 de 0,355 contra 0,304.

Mostre também que as métricas binárias seguem o enunciado: relevante é grau maior ou igual a 1, e tanto -1 quanto não julgado contam como não relevante.

experiments.py: um experimento por requisito

Cada função `experiment_*` corresponde a um requisito, grava seus arquivos em `results/` e devolve o que o relatório precisa.

Função	Requisito	O que produz
<code>experiment_preprocessing</code>	1	Tabela 1, Figura 1
<code>experiment_models</code>	2, 3, 4, 5	Tabela 2, Figura 2, as 225 linhas por consulta
<code>experiment_query_analysis</code>	6	Tabela 3, as seis consultas com Top-5
<code>experiment_bm25_parameters</code>	7	Tabelas 4 e 5, Figura 4

Função	Requisito	O que produz
<code>experiment_query_reformulation</code>	8	Tabelas 6 e 7
<code>experiment_error_analysis</code>	9	a seção 4.7, com decomposição de score
<code>experiment_length_analysis</code>	apoio ao 5	Figura 3, o teste da tese
<code>experiment_design_choices</code>	apoio	Robertson contra BIM, ganho invertido contra bruto

O ponto metodológico mais forte da sua fala: **a seleção das consultas é automática e determinística, não escolhe a dedo depois de ver os resultados**. No requisito 6, entre as consultas com pelo menos 4 relevantes, o código toma as duas de maior `delta_AP`, as duas de menor e as duas de menor AP médio. No requisito 9, os falsos positivos são os não relevantes que o BM25 põe nas duas primeiras posições, e o falso negativo é o relevante de pior posto.

Diga também que **os qrels só são usados para avaliar**. Nenhuma etapa de indexação, ponderação, escolha de parâmetros ou reformulação os consulta, como o enunciado exige.

Requisito 5: a média esconde o essencial

O BM25 ganha em todas as seis métricas, com vantagem maior no MRR (+10,7%) e no MAP (+10,3%). Mas a frase que vale é a seguinte: **o BM25 vence em 137 consultas, perde em 81 e empata em 7**. A pergunta interessante não é qual modelo tem a maior média, e sim o que separa os dois grupos.

Requisito 6: a leitura dos Top-5

Na Tabela 3, leia os pares lado a lado e procure comprimento de documento. Em q223 o vetorial põe no topo os documentos 1008 (51 tokens) e 864 (29 tokens), ambos irrelevantes e curtos; o BM25 acha os relevantes 1398 (115 tokens) e 1387 (120 tokens). Em q113 o sinal se inverte: o BM25 traz 704 (185), 815 (142) e 638 (138), todos longos e irrelevantes, enquanto o vetorial acerta com 265 (49) e 748 (83).

O par q124 e q13 é de outra natureza e vale guardar: em q13 o documento 496 tem casamento léxico quase perfeito com a consulta e mesmo assim foi julgado -1. O erro não está no ranqueamento, mas no descompasso entre sobreposição léxica e relevância julgada.

Requisito 7: b importa mais que k1

A grade cobre k1 em 0,5, 1,2 e 2,0 e b em 0, 0,75 e 1. O melhor ponto é k1 = 2,0 com b = 0,75, MAP 0,312.

O resultado para enfatizar: **fixado k1, ir de b = 0 para b = 0,75 melhora o MAP entre 8,6% e 10,8%; variar k1 inteiro com b fixo rende no máximo 7%**. Isso é coerente com a tese do trabalho.

Há ainda interação entre os parâmetros: para k1 pequeno o melhor b é 1,0, mas para k1 = 2,0 o melhor passa a 0,75. A explicação é que k1 alto deixa o score mais sensível à frequência bruta, e aí normalizar integralmente passa a penalizar demais documentos longos com ocorrências legítimas.

A Tabela 5 é o exemplo concreto que o enunciado pede. Na q67, com b = 0 o Top-5 é tomado por documentos de 177, 164, 138, 130 e 116 tokens, só um relevante; com b = 1 os cinco são relevantes e têm 17 a 91 tokens. A AP vai de 0,14 para 0,55.

Requisito 8: as cinco reformulações

Em `query_reformulations.py`, cada uma das cinco tem a hipótese escrita no campo `hypothesis` **antes** de rodar os modelos. Isso é importante dizer: a hipótese veio primeiro, o resultado depois.

Consulta	Operação	O que ensina
q219	sinônimo, <code>small</code> por <code>low</code>	o caso mais limpo: AP do vetorial de 0,045 para 0,119
q69	mais específica	termos pouco discriminativos diluem a consulta
q109	consulta curta expandida	4 tokens não separam documentos
q171	remover termos	hipótese parcialmente refutada
q15	vocabulário técnico	a AP não vê a melhora, o NDCG vê

Dois casos merecem destaque na fala. Em **q171** prevíamos que enxugar a consulta ajudaria sobretudo o vetorial; ele melhorou pouco (0,306 para 0,319) mas o BM25 piorou (0,639 para 0,556). A explicação é que o BM25 não normaliza o vetor de consulta, então remover termos é descartar evidência sem contrapartida. Admitir uma hipótese refutada conta a favor.

Em **q15** a AP do BM25 não muda (1,00 para 1,00) mas o NDCG@10 sobe de 0,74 para 1,00, porque a reformulação trouxe para a 1ª posição o documento de grau 1. É o argumento a favor de reportar métrica graduada.

Requisito 9: três tipos de erro

Use a decomposição de score que o `explain()` fornece, porque ela permite argumentar sobre a causa em vez de especular.

1. **Independência entre termos falhando.** Na q98 o documento 638 vem em 1º com score 35,1, somando `flap` (+6,74), `control` (+5,70), `trail` (+4,87) e `subson` (+3,03). Todos os termos estão lá, mas a consulta pergunta por flaps de bordo de fuga e o documento trata de flaps de nariz.
2. **Limitação dos qrels.** Na q139 os documentos 847 e 897 parecem pertinentes mas não foram julgados. Como o enunciado manda tratar não julgado como não relevante, contam como erro do sistema sem o serem.
3. **O limite do casamento léxico.** Na q80 o documento 596 tem grau 3 e aparece na posição 1206 nos dois modelos, porque a interseção entre os tokens da consulta e os do documento é vazia. Nenhum ajuste de k_1 , b ou pré-processamento recupera isso.

plots.py e os testes

`plots.py` gera as oito figuras com uma paleta categórica consistente. Se perguntarem, a escolha de cor por função: matiz distinto para séries categóricas, rampa de matiz único para magnitude contínua na Figura 4.

Os 16 testes em `tests/test_ir_tp1.py` cobrem três frentes, e vale citá-los como prova de correção:

1. **Métricas contra cálculo manual**, usando o exercício do slide 41 da Aula 05.
2. **Implementação de referência ingênua**: uma versão em Python puro com laços diretos das fórmulas reproduz os scores da versão vetorizada em SciPy com tolerância $1e-10$, nos dois modelos e em três combinações de (k_1, b) .
3. **Verificação cruzada com scikit-learn**: correlação de Spearman acima de 0,95 com o `TfidfVectorizer`. Não se espera igualdade exata porque o scikit-learn usa `idf = \ln(N/df) + 1`.

Há ainda `test_ranking_tie_break_is_deterministic`, `test_query_qrel_alignment` e `test_bm25_with_k1_zero_is_binary`, que testam exatamente as três decisões mais delicadas do trabalho.

A tese que une as duas partes

Esta é a seção que os dois precisam saber de cor, porque é o que transforma uma lista de experimentos em um argumento.

A afirmação. O cosseno divide pela norma completa do vetor, o que penaliza integralmente um documento longo, mesmo quando o comprimento vem de tratar o assunto com profundidade. O BM25 com $b = 0,75$ normaliza apenas parcialmente. Quase toda a discordância entre os dois modelos nesta coleção sai daí.

Como ela foi construída. A hipótese nasceu da inspeção de seis consultas no requisito 6. Seis exemplos não provam nada, então o `experiment_length_analysis` a testou sobre as 225 consultas, com duas previsões verificáveis.

Previsão	Resultado	Veredito
O Top-10 do BM25 deve ser mais longo que o do vetorial	98,4 contra 79,4 tokens	confirmada
O BM25 deve ganhar onde os relevantes são longos	Spearman rho = 0,432, p da ordem de $1e-11$, n = 225	confirmada

O detalhe que fecha o argumento. O comprimento médio da coleção é 92,1 tokens e o dos documentos relevantes é 92,5, praticamente idêntico. Ou seja, ser longo não torna um documento relevante. Ainda assim o Top-10 do vetorial tem 79,4 tokens, bem abaixo da média da coleção. **O vetorial tem um viés sistemático para documentos curtos que os dados não justificam.**

Onde a tese reaparece. Vale apontar cada uma dessas conexões durante a fala, porque elas mostram que os experimentos conversam entre si:

- No requisito 1, as stopwords cortam 42% dos tokens e mexem no denominador das normalizações.
- No requisito 6, os seis Top-5 se explicam por comprimento.
- No requisito 7, b domina k_1 , e b é exatamente o parâmetro de normalização.
- No requisito 8, q171 revela a assimetria: o BM25 não normaliza o vetor de consulta.

O limite da tese. Ela explica a discordância entre os modelos, não o teto dos dois. As consultas em que ambos falham falham por outro motivo: ou o relevante não compartilha nenhum termo com a consulta (q80), ou a consulta é composta (q124), ou o julgamento

não acompanha a sobreposição léxica (q13). Trocar `small` por `low` em q219 quase triplicou a AP, ganho que nenhum ajuste de parâmetro produziria. É onde entram expansão de consulta e representações densas.

Materiais gerados e de onde vem cada número

Se pedirem a origem de algum valor do relatório, esta tabela responde. Nenhum número foi digitado à mão: todos saem de `results/`, que tem 25 arquivos.

Elemento do relatório	Arquivo que o produz
Tabela 1, efeito do pré-processamento	<code>01_preprocessing_summary.csv</code> , <code>01_preprocessing_vocab.csv</code>
Tabela 2, métricas agregadas	<code>02_model_summary.csv</code>
Figura 2, curva P x R	<code>02_precision_recall_curves.json</code>
Tabela 3, Top-5 das seis consultas	<code>03_query_analysis.json</code>
Figura 3, AP por consulta	<code>02_model_comparison_per_query.csv</code>
Tabela 4, grade k1 x b	<code>04_bm25_parameter_grid.csv</code>
Tabela 5, efeito de b em q67	<code>04_effect_of_b_showcase.json</code>
Tabelas 6 e 7, reformulação	<code>05_query_reformulation.csv</code> , <code>05_query_reformulation_detail.json</code>
Seção 4.7, análise de erros	<code>06_error_analysis.json</code>
Seção 4.4, teste da tese	<code>08_length_analysis_summary.json</code>
Robertson contra BIM	<code>07_idf_variants.csv</code>

O requisito 3 da entrega pede resultados por consulta, e eles existem para cada experimento: `01_preprocessing_per_query.csv` com 1800 linhas, `02_per_query_metrics.csv`, `04_bm25_parameter_per_query.csv` com 2025 linhas e `08_length_analysis_per_query.csv`.

Reprodutibilidade

Se perguntarem se o resultado é reproduzível, a resposta é demonstrável: `python run_all.py --no-download` roda tudo em cerca de 15 segundos e os arquivos saem byte a byte idênticos. Não há aleatoriedade no pipeline, e empates de score são desfeitos pelo `doc_id` crescente.

O SHA-256 do tarball do Cranfield é conferido no download, então a coleção também é a mesma.

Figuras

São oito PNGs em `figures/`, dos quais quatro entraram no relatório: `fig1b` (pré-processamento), `fig3` (curva P x R), `fig5` (AP por consulta) e `fig4_MAP` (grade de parâmetros). As outras quatro ficaram de fora por limite de páginas, mas estão na entrega.

Perguntas prováveis da defesa

O enunciado diz que qualquer integrante pode ser chamado a explicar uma parte do código, interpretar um resultado, justificar o score de um documento ou prever o efeito de mudar um parâmetro. As perguntas abaixo seguem esse texto.

Sobre fórmulas

Por que o cosseno vira produto interno no código? Porque normalizamos os vetores de documento para norma L2 unitária na indexação e o vetor de consulta também é normalizado em `query_weights`. Com os dois unitários, o denominador do cosseno vale 1.

O que acontece com $k1 = 0$? O quociente vira $(0+1)/(0+K)$ multiplicado por `f`, que se reduz a uma indicadora: só importa se o termo ocorre, não quantas vezes. O modelo fica binário, e `test_bm25_with_k1_zero_is_binary` verifica isso.

E com $b = 0$ e $b = 1$? Com $b = 0$ não há normalização nenhuma e documentos longos acumulam ocorrências; com $b = 1$ a normalização é integral. A Tabela 5 mostra os dois extremos na q67.

Por que o idf de um termo presente em todos os documentos é zero? Porque $\log(N/N) = \log(1) = 0$. Na prática o termo deixa de discriminar e é eliminado do ranking.

Sobre decisões de implementação

Por que remover stopwords antes do stemming? A lista do NLTK é definida sobre formas plenas. Se stemizarmos antes, o radical de uma stopword pode coincidir com o de um termo de conteúdo e a remoção passa a derrubar o termo errado.

Por que não concatenar sempre título e resumo? Porque em 1395 dos 1400 documentos o campo `.w` já começa com o título. Concatenar dobraria o `tf` dos termos do título sem razão.

Por que guardar a matriz em CSR e CSC? CSR dá acesso barato ao vetor de um documento, usado no vetorial; CSC dá acesso barato à lista de postings de um termo, que é como o BM25 percorre a coleção.

Como vocês garantem que a comparação entre os modelos é justa? Os dois derivam seus pesos da mesma matriz de frequências brutas, construída uma única vez. A única coisa que muda entre eles é a função de ranqueamento.

Sobre avaliação

Por que o denominador da AP é o total de relevantes? Para que relevantes não recuperados entrem como zero. Se dividíssemos pelos recuperados, uma consulta que achou 1 de 20 relevantes na 1ª posição teria $AP = 1,0$, o que seria enganoso.

Por que inverter a escala do Cranfield no NDCG? Porque grau 1 é o melhor na convenção da coleção. Usar o grau bruto como ganho daria peso maior ao documento menos relevante. O efeito é grande: 0,355 contra 0,304.

Vocês usaram os qrels para ajustar alguma coisa? Não. Eles entram só na avaliação. A escolha do pré-processamento vencedor, dos parâmetros e das consultas analisadas é feita por critério determinístico documentado em `experiments.py`.

Como sabem que as métricas estão certas? Foram conferidas contra o cálculo manual do slide 41 da Aula 05, e há uma implementação ingênua em Python puro que reproduz os scores da versão vetorizada com tolerância $1e-10$.

Sobre resultados

Se eu aumentar k_1 de 1,2 para 2,0, o que acontece? O MAP sobe de 0,305 para 0,312 com $b = 0,75$. Mas o melhor b muda junto, de 1,0 para 0,75, porque k_1 alto deixa o score mais sensível à frequência bruta.

Por que o documento 638 aparece em 1º na q98 se não é relevante? A decomposição do score mostra `flap` (+6,74), `control` (+5,70), `trail` (+4,87) e `subson` (+3,03). Todos os termos da consulta estão no documento, mas a consulta pede uma relação entre eles que o modelo não enxerga, porque assume independência.

Por que o BM25 perde em 81 consultas se é melhor na média? Porque a normalização parcial é uma aposta. Paga bem quando os relevantes são longos e custa caro quando são curtos, como em q113 e q190.

Algum resultado contrariou a expectativa de vocês? Sim, q171. Prevíamos que remover termos ajudaria o vetorial e o BM25 piorou. Descobrimos que o BM25 não normaliza o vetor de consulta, então termos a mais só somam evidência.

Se não souberem responder

Digam onde o código responde, em vez de inventar. Os métodos `explain()` do `vsm.py` e do `bm25.py` decompõem qualquer score termo a termo, e `results/` guarda o resultado de cada consulta.